

接口模型管理

原语雀文档链接: <https://yuque.antfin.com/amap-search/nen549/vdmwa69qe4eleg96>

前期分析

参考: [US 标准输出模型文档](#)

背景简介

US服务接入GBF工作量最大的内容之一就是模型的统一维护；尤其是各方共建的领域模型，其字段和结构复杂，迭代变更频繁，开发过程面临腐化失控的挑战；因此，基于框架实现的统一模型管控凭条成为非常迫切的诉求。

目标收益

- 模型
 - 统一管控，避免重复修改建设，避免分歧
 - 内容可视，更容易理解其业务含义和引用关系
 - 数据验证，用于验证数据的合规性
- 接口
 - 基于框架透出接口的文档，用于上下游交互
 - 基于框架透出接口的mock功能，用于上下游并行开发联调

需求分析

问题场景

- 架构委员会定义了统一的领域模型，实际业务开发时可能随意改变或增加重复属性，打破领域模型的统一标准
 - 需要能直观感知到有人打破了规则
 - 需要方便地让业务开发同学看到统一的标准
- 进行版本迭代的时候，上下游需要知道业务服务所提供的接口及出入参模型
 - 以前靠手写文档，且无mock能力
 - 需要有自动的代码生成api文档的能力
 - 需要有并行开发时的mock能力
- 模型和专业词需要有统一的wiki库来对所有接入人员进行认知拉齐

- 模型属性描述
- 模型引用关系
- 模型校验功能
- 业务专业词库

功能需求

1. GBF进行业务改造和领域建设过程中，模型统一、规范、可视。
 - a. 领域模型、领域事件、专业词库
 - b. 模型规范、模型校验、模型版控？
 - c. 模型引用、词库关系
2. 上下游交互有统一便捷的接口文档可见。
 - a. 接口地址、路径、参数
 - b. 模型引用、词库关系
 - c. mock功能？

技术需求

- 模型
 - 模型管理
 - 导入 (json、pojo)
 - 导出 (json、pojo)
 - 模型解释
 - 类javadoc文档？
 - 自定义注解识别
 - 模型编排
 - 识别领域模型与子域模型引用关系
 - 复杂聚合逻辑
 - 键值聚合
 - 列表聚合
 - 模型校验
 - 校验注解
 - 校验引擎
 - 校验执行
 - 模型驱动
- 接口
 - 接口文档
 - 复用现有平台？
 - 关联模型
 - 参数注释
 - 接口mock

实现分析

前期调研

产品调研

[产品调研](#)

技术调研

[技术调研](#)

差异分析

产品差异

调研的产品主要集中在下面两类

- API管理
- 模型和数据驱动

我们的产品诉求在二者中间，需要

- api管理
- 模型管理

这两个能力，所以不能直接复用调研产品，而是参考其做法实现自己的产品。

技术差异

技术调研主要分析了

- 集团内的一些元数据驱动架构
- JsonSchema标准以及和POJO互转的技术
- Swagger方案以及对应的OpenApi标准
- JavaDoc和其他Api管理平台的等

通过调研分析，发现这些技术与我们的诉求契合度最高的是JsonSchema标准，它可以完整地表述我们定义的POJO模型标准，同时也有完善的开源功能实现，可以在代码模型和Json定义间互转；

进一步调研发现，Swagger以及它所用的OpenApi标准实际是JsonSchema的超集，它除了定义自己的apiJson标准，api所用的Schema模型都按照JsonSchema（Draft5）来定义的，因此Swagger的方案优于单纯的JsonSchema。

进一步我们对可行的几个技术方案进行各方面的综合分析，评价如下：

方案	收益				成本				
	适用性	三高	可扩展性	可维护性	建设成本	使用成本	维护成本	总分	收益
	30.0%	20.0%	5.00%	10.0%	10.0%	15.00%	10.0%		按比

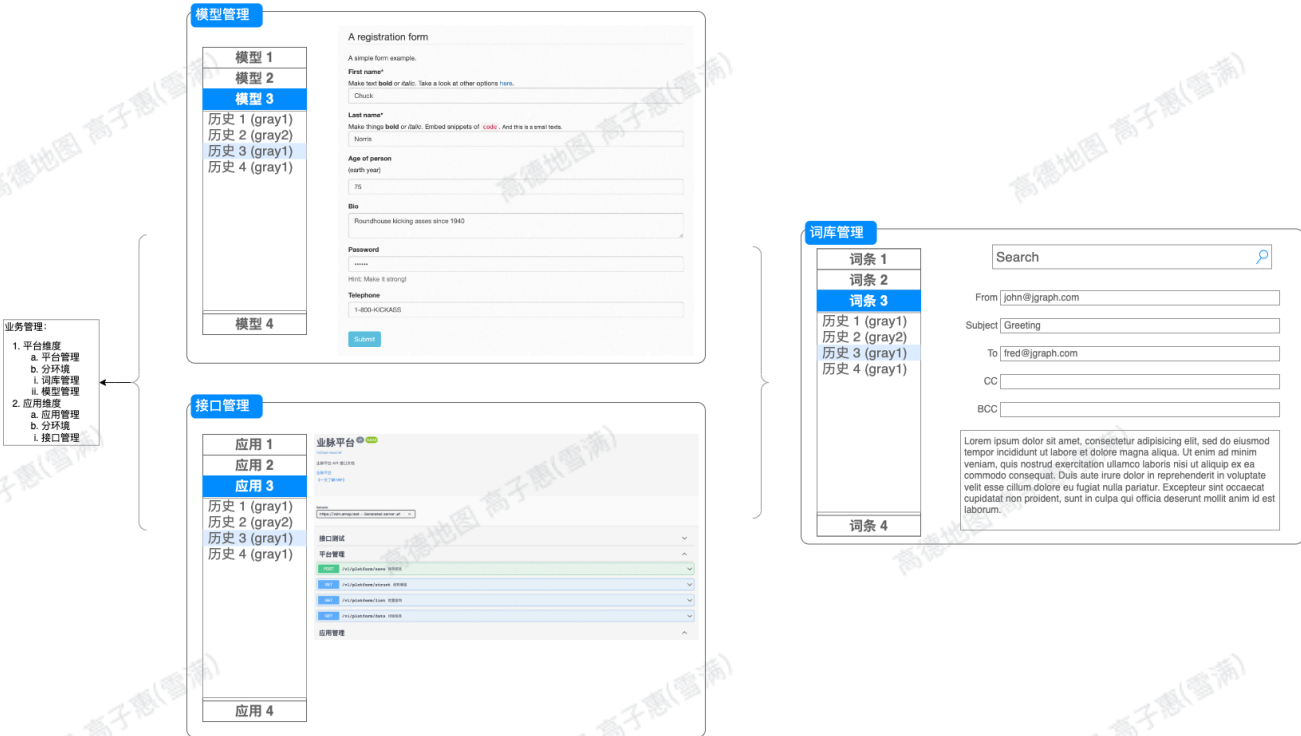
JsonSchema	2	3	4	3	3	3	3	2.75	2.62
Swagger	3	4	2	3	4	4	4	3.5	3.25
自研	4	3	4	4	2	2	2	3.1	3.65

实现结论

通过上面分析，决定采用Swagger的方案来实现框架的api+模型数据的识别采集，同时平台产品测也复用一些Swagger生态的开源组件来实现前端的显示、修改能力。

方案设计

功能规划



业务管理：

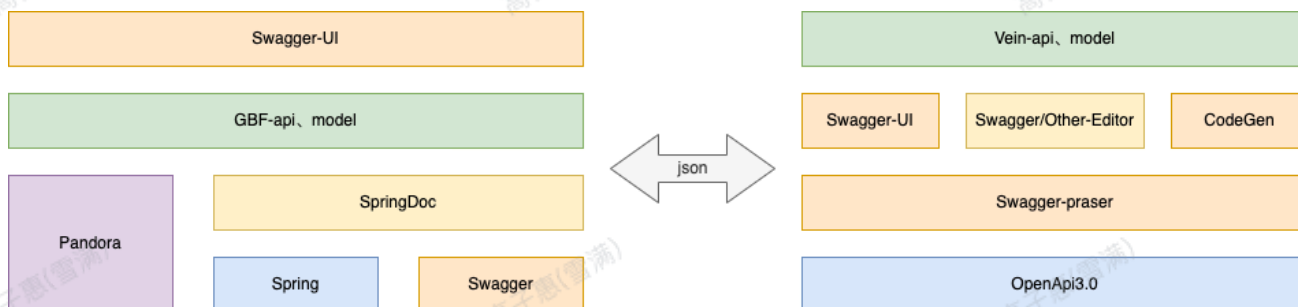
1. 平台维度
 - a. 平台管理
 - b. 分环境
 - i. 词库管理
 - ii. 模型管理
2. 应用维度
 - a. 应用管理

b. 分环境

i. 接口管理

此外：根据对业脉的菜单布局进行调整，将管理菜单与数据菜单拆分开，显示更清晰合理。

整体方案

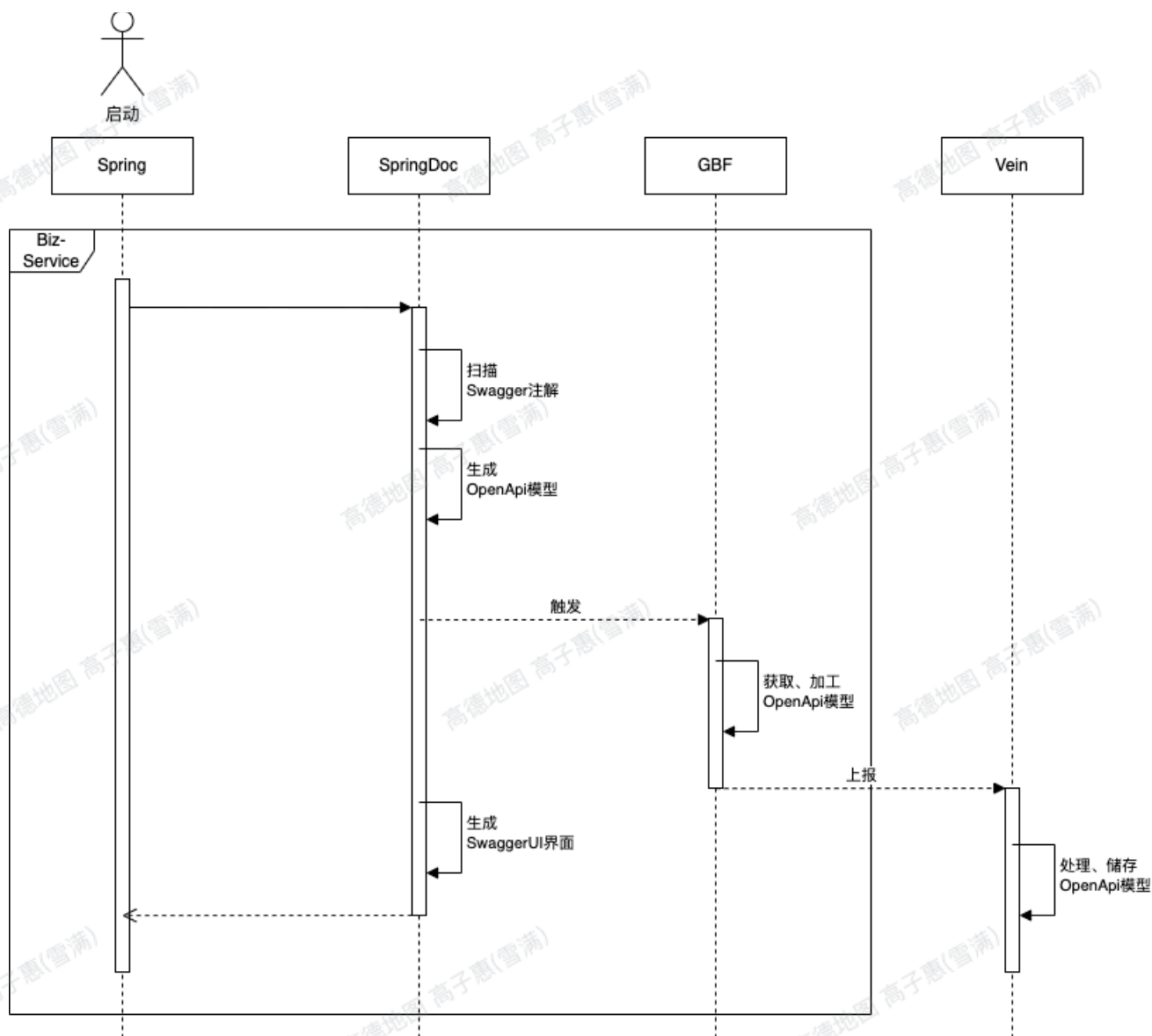


说明：

- GBF集成SpringDoc提供OpenapiApi标准数据生成能力
- GBF将生成的OpenApi数据上报到业脉平台实现管理和可视化
- 业脉平台对收到的上报数据进行解析入库
 - Api数据
 - 模型数据
 - 词库数据
- 业脉平台集成OpenApi生态的开源组件实现产品能力
 - 解析：Swagger-parser
 - Api可视：Swagger-UI
 - 模型编辑/可视：Swagger-Editor或其他表顺JsonSchema编辑器
 - Json转代码：swagger-codegen、autorest、jsonschema2pojo
 - 代码转Json：Swagger-Core、jsonschema-generator
 - 综合解析：openapi-generator (client、server、doc...)

关键方案

服务启动



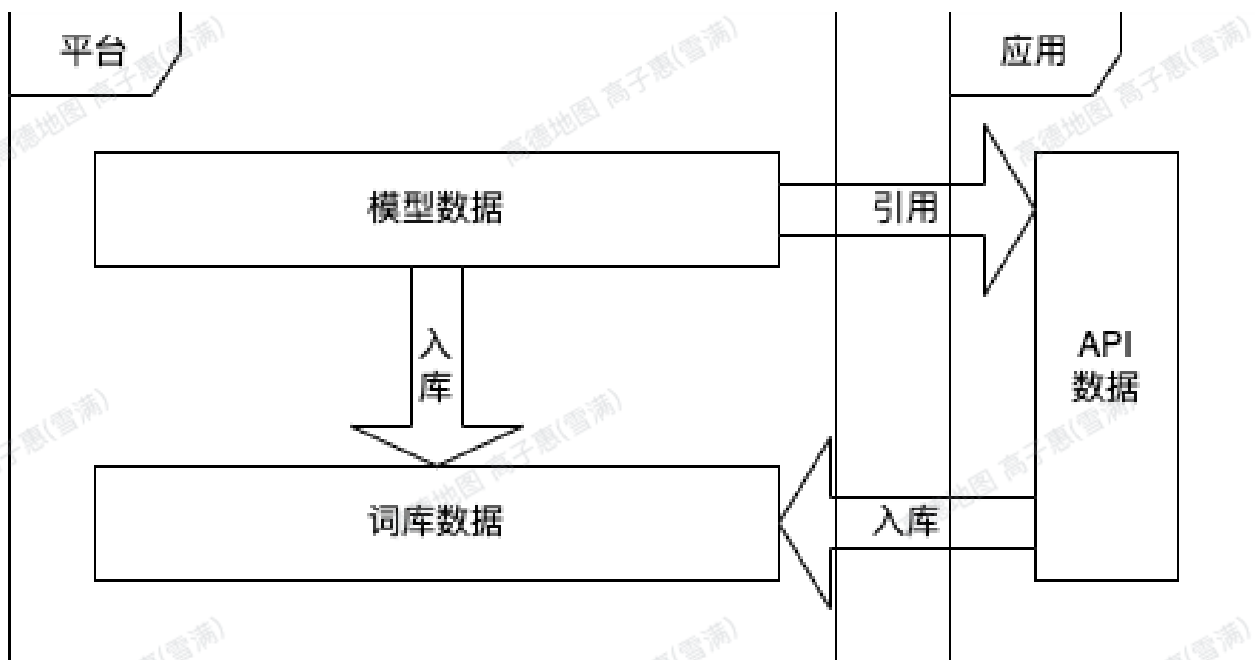
原则：

- 服务启动后数据上报过程异步解耦

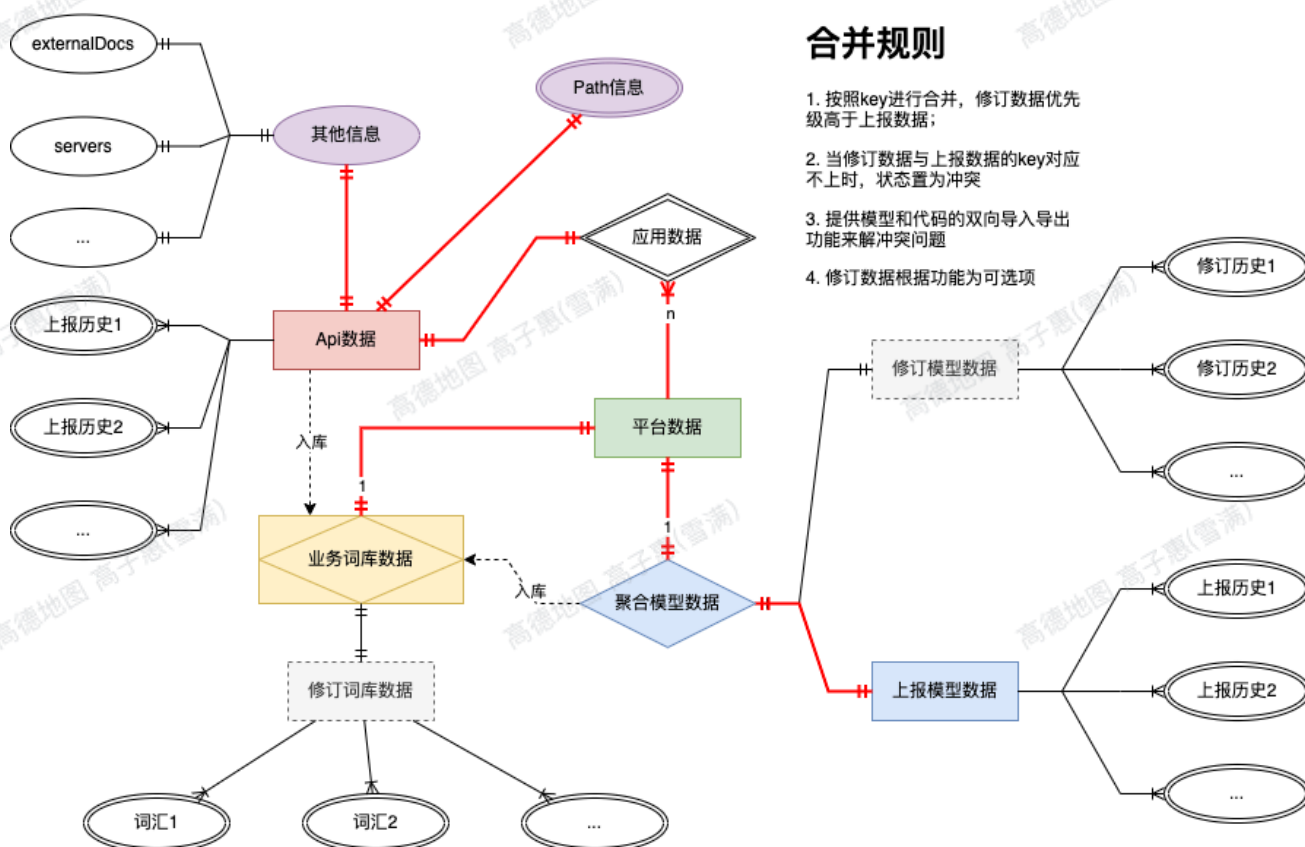
数据模型

模型设计

1. 模型关系



2. 模型结构



合并规则

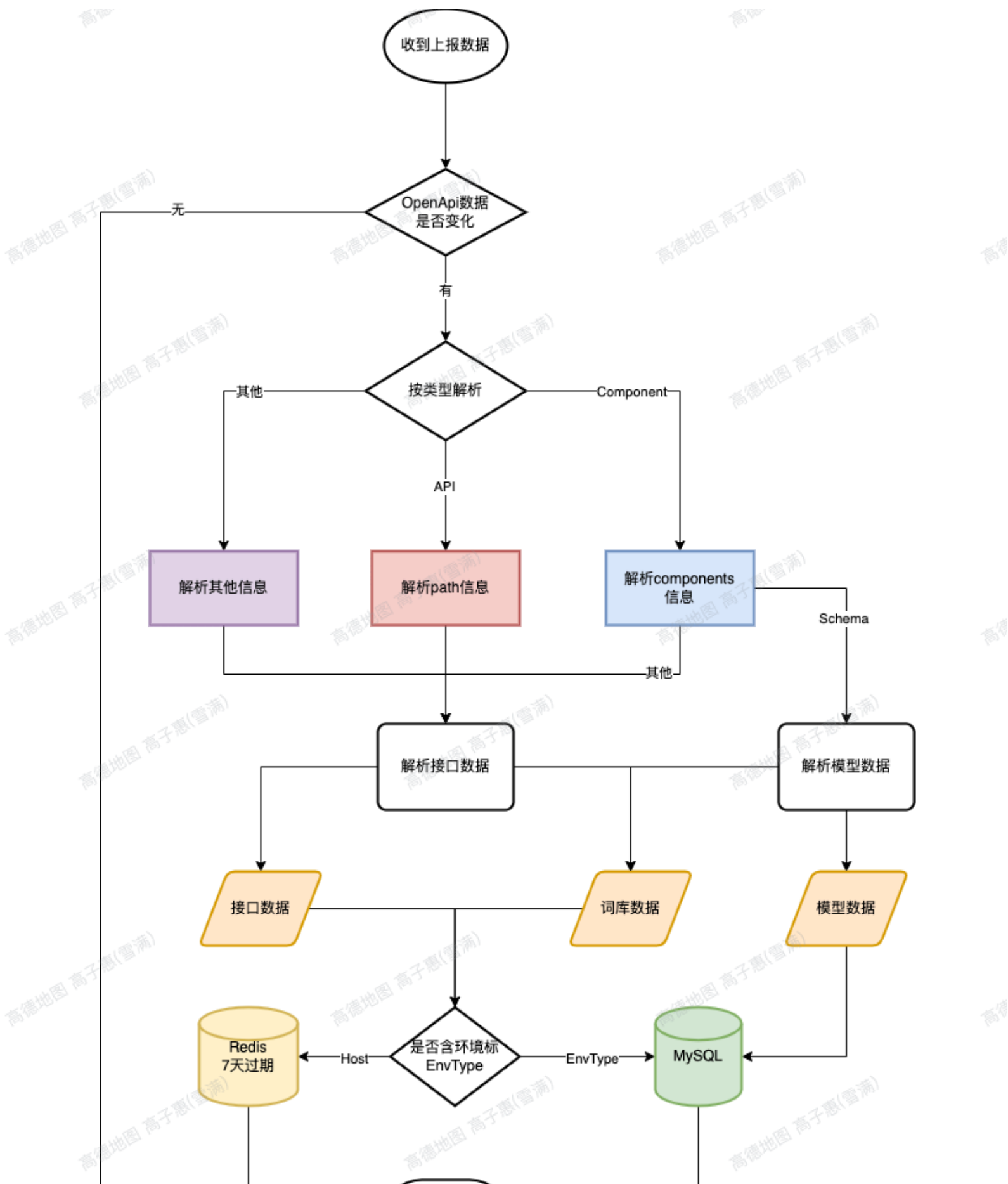
1. 按照key进行合并，修订数据优先级高于上报数据；
2. 当修订数据与上报数据的key对应不上时，状态置为冲突
3. 提供模型和代码的双向导导入导出功能来解决冲突问题
4. 修订数据根据功能为可选项

说明：

1. 一个平台包含如下数据
 - a. 一套平台数据
 - i. 一套业务词库数据
 - ii. 一套模型数据
 - b. 多套应用数据

- i. 一套path数据
 - ii. 一套其他数据
- 2. 所有数据中api和模型数据包括历史记录数据，其余数据为单条覆盖储存
- 3. 业务词库数据由多个数据来源聚合而成
 - a. api参数
 - b. 模型属性
 - c. 人工修订？
- 4. 数据模型中考虑对模型和词库提供修订能力？
 - a. 根据字段名（jsonKey）进行聚合
 - b. 根据聚合结果提供修订状态，当代码模型不同于平台所修订标准时能提供冲突提示
 - c. 此功能为设计预留的可选能力，根据需要再实现

模型管理





说明：

1. Doc数据和元数据拆分，单独上报
2. 业脉根据应用上报的数据进行分类解析
 - a. api数据与其他数据
 - i. 按应用归类存储
 - ii. 保留多条历史数据
 - iii. 提供修订聚合逻辑
 - iv. 参数解析入业务词库
 - b. 模型数据
 - i. 按平台统一储存
 - ii. 保留多条历史数据
 - iii. 提供修订聚合逻辑
 - iv. 属性解析入业务词库
 - c. 词库数据
 - i. 按平台统一储存
 - ii. 保留多条历史数据
3. 上报数据会与平台的修订数据进行聚合，前端显示聚合后的数据
 - a. 聚合失败后返回冲突警告到前端，提示用户解决一致性问题
 - b. 可修订数据包含：~~api数据、模型数据、业务词库~~
 - c. 此功能为设计预留的可选能力，~~根据需要再实现~~

前端组件选型

根据产品和技术调研，前端要实现接口、模型的界面功能有如下两种方案，具体根据前端开发来决定：

方案	优点	缺点	备注
swagger (ui+editor)	- 接口和模型均支持编辑 - 数据兼容性好 - 能建立模型和数据关系	- 界面布局单一不利于阅读 - 非交互式编辑	除了界面友好度ok
开源组合 (redoc+react-jsonschema-form/jsonforms)	- 界面显示合理清晰 - 交互式编辑	- 数据兼容性一般 - 无api编辑能力 - 无识别关联关系能力	需要有一定二次开发

详细设计

GBF

Openapi集成

复制代码

```
/**
 * 注册Doc
 * @author wangyalei
 */
@Component
public class DocRegister implements Register {

    @Override
    public void doRegister() {
        try {
            DocResource docResource =
                BeanUtil.getSpringBean(DocResource.class);
            if (null == docResource) {
                // 不符合配置条件或没开Doc
                LaunchLog.warn("DocResource没有加载，将跳过Doc注册！");
                return;
            }

            // 注册DocResource
            DocRegistry.setDocResource(docResource);

            // 注册成功
            LaunchLog.info("DocResource注册成功！");
        } catch (Throwable t) {
            throw new RegistryException("DocResource注册失败！", t);
        }
    }
}
```

数据处理上报

复制代码

```

public static void updateDoc(OpenAPI data) {
    String raw = null;
    try {
        SyncDTO<OpenAPI> sync = SyncDTO.of(data);
        raw = ToString.toJsonJSON(sync);
        String tags = "Doc";
        Message message = new Message(TOPIC, tags, raw.getBytes());
        SendResult rs = PRODUCER.send(message);
        LaunchLog.info("DocData send success TOPIC=" + TOPIC + ",tags=" +
tags + ",msgId=" + rs.getMsgId()
+ ",sdkVersion=" + SDK_VERSION + ",commitId=" + COMMIT_ID);
    } catch (Exception e) {
        LaunchLog.error(e, "DocData send fail ,data=" + raw +
",sdkVersion=" + SDK_VERSION + ",commitId=" + COMMIT_ID);
    }
}

```



复制代码

```

@Data
public class SyncDTO<T> {
    /**
     * 所属平台
     */
    private String system;
    /**
     * 所属应用
     */
    private String appName;
    /**
     * git提交id
     */
    private String commitId;
    /**
     * 环境标
     */
    private String envType;
    /**
     * sdk版本
     */
    private String sdkVersion;

    /**
     * 传输数据
     */
}

```

```
private T data;

public static <D> SyncDTO<D> of(D data) {
    SyncDTO<D> syncDTO = new SyncDTO<>();
    syncDTO.setSystem(Properties.getPlatformCode());
    syncDTO.setAppName(AppNameUtils.getAppName());
    syncDTO.setCommitId(RegistryUploader.COMMIT_ID);
    syncDTO.setEnvType(RegistryUploader.ENV_TYPE);
    syncDTO.setSdkVersion(RegistryUploader.SDK_VERSION);
    syncDTO.setData(data);
    return syncDTO;
}
}
```

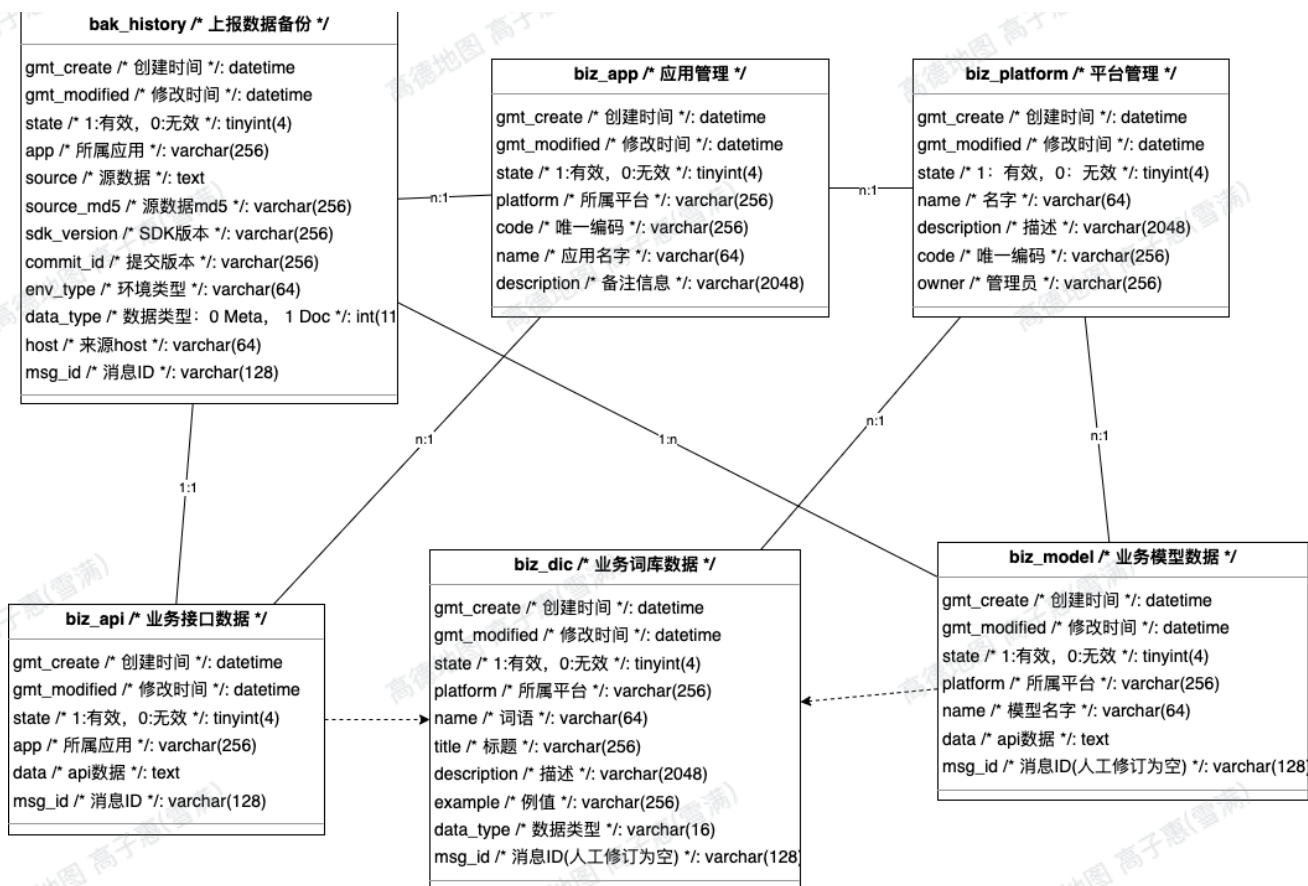
业脉

数据接收

todo

数据管理

数据库



复制代码

```

/*****
/*   DatabaseName = vein_pre   */
/*   TableName = bak_history   */
*****/
CREATE TABLE `bak_history`
(
    `id`          bigint(20) unsigned NOT NULL AUTO_INCREMENT COMMENT '主键',
    `gmt_create`  datetime              DEFAULT CURRENT_TIMESTAMP
    COMMENT '创建时间',
    `gmt_modified` datetime              DEFAULT CURRENT_TIMESTAMP ON
    UPDATE CURRENT_TIMESTAMP COMMENT '修改时间',
    `state`       tinyint(4)            NOT NULL DEFAULT '1' COMMENT '1:有效, 0:
    无效',
    `app`         varchar(256)           NOT NULL COMMENT '所属应用',
    `source`      text                   NOT NULL COMMENT '源数据',
    `source_md5`  varchar(256)           DEFAULT NULL COMMENT '源数据
    md5',
    `sdk_version` varchar(256)           DEFAULT NULL COMMENT 'SDK版本',
    `commit_id`   varchar(256)           DEFAULT NULL COMMENT '提交版本',
    `env_type`    varchar(64)            DEFAULT NULL COMMENT '环境类型',

```

```

        `data_type`      int(11)                NOT NULL DEFAULT '0' COMMENT '数据类型:
0 Meta, 1 Doc',
        `host`           varchar(64)             DEFAULT NULL COMMENT '来源
host',
        `msg_id`         varchar(128)            DEFAULT NULL COMMENT '消息ID',
        PRIMARY KEY (`id`),
        UNIQUE KEY `uk_bak_history` (`app`, `msg_id`, `source_md5`)
    ) ENGINE = InnoDB
    DEFAULT CHARSET = utf8mb4 COMMENT ='上报数据备份'
;

```

```

/*****
/* DatabaseName = vein_pre */
/* TableName = biz_api */
*****/
CREATE TABLE `biz_api`
(
    `id`                bigint(20) unsigned NOT NULL AUTO_INCREMENT COMMENT '主键',
    `gmt_create`        datetime              DEFAULT CURRENT_TIMESTAMP
COMMENT '创建时间',
    `gmt_modified`      datetime              DEFAULT CURRENT_TIMESTAMP ON
UPDATE CURRENT_TIMESTAMP COMMENT '修改时间',
    `state`             tinyint(4)           NOT NULL DEFAULT '1' COMMENT '1:有效, 0:
无效',
    `app`               varchar(256)         NOT NULL COMMENT '所属应用',
    `data`              text                 NOT NULL COMMENT 'api数据',
    `msg_id`            varchar(128)         DEFAULT NULL COMMENT '消息ID',
    PRIMARY KEY (`id`),
    UNIQUE KEY `uk_biz_api` (`app`, `msg_id`)
) ENGINE = InnoDB
DEFAULT CHARSET = utf8mb4 COMMENT ='业务接口数据'
;

```

```

/*****
/* DatabaseName = vein_pre */
/* TableName = biz_model */
*****/
CREATE TABLE `biz_model`
(
    `id`                bigint(20) unsigned NOT NULL AUTO_INCREMENT COMMENT '主键',
    `gmt_create`        datetime              DEFAULT CURRENT_TIMESTAMP
COMMENT '创建时间',
    `gmt_modified`      datetime              DEFAULT CURRENT_TIMESTAMP ON
UPDATE CURRENT_TIMESTAMP COMMENT '修改时间',

```

```

        `state`                tinyint(4)                NOT NULL DEFAULT '1' COMMENT '1:有效, 0:
无效',
        `platform`            varchar(256)                NOT NULL COMMENT '所属平台',
        `name`                 varchar(64)                NOT NULL COMMENT '模型名字',
        `data`                 text                        NOT NULL COMMENT 'api数据',
        `msg_id`               varchar(128)                DEFAULT NULL COMMENT '消息ID(人
工修订为空)',
        PRIMARY KEY (`id`),
        UNIQUE KEY `uk_biz_model` (`platform`, `name`, `msg_id`)
) ENGINE = InnoDB
DEFAULT CHARSET = utf8mb4 COMMENT ='业务模型数据'
;

/*****
/* DatabaseName = vein_pre */
/* TableName = biz_dic */
*****/
CREATE TABLE `biz_dic`
(
    `id`                bigint(20) unsigned NOT NULL AUTO_INCREMENT COMMENT '主键',
    `gmt_create`        datetime                DEFAULT CURRENT_TIMESTAMP
COMMENT '创建时间',
    `gmt_modified`      datetime                DEFAULT CURRENT_TIMESTAMP ON
UPDATE CURRENT_TIMESTAMP COMMENT '修改时间',
    `state`             tinyint(4)                NOT NULL DEFAULT '1' COMMENT '1:有效, 0:
无效',
    `platform`          varchar(256)                NOT NULL COMMENT '所属平台',
    `name`              varchar(64)                NOT NULL COMMENT '词语',
    `title`             varchar(256)                DEFAULT NULL COMMENT '标题',
    `description`        varchar(2048)              DEFAULT NULL COMMENT '描述',
    `example`           varchar(256)                DEFAULT NULL COMMENT '例值',
    `data_type`         varchar(16)                NOT NULL DEFAULT 'string' COMMENT '数据
类型',
    `msg_id`            varchar(128)                DEFAULT NULL COMMENT '消息ID(人
工修订为空)',
    PRIMARY KEY (`id`),
    UNIQUE KEY `uk_biz_dic` (`platform`, `name`, `msg_id`)
) ENGINE = InnoDB
DEFAULT CHARSET = utf8mb4 COMMENT ='业务词库数据'

```

数据解析

数据存取

数据与代码转换

接口mock

todo

前端

总体

词库

模型

接口

其他

实施计划

-

当前遗留

-

后续规划

-

附录

模型分类

- 战略建模
 - 领域 (Domain) : 即一个组织所做的事情及其所包含的一切。核心在于识别不同的域, 确定域的职责。
 - 限界上下文 (Bounded Context) : 是一个显示边界, 领域模型存在于边界内。确定领域边界及上下文。
 - 上下文映射 (Context Map) : 确立上下文之间的关系, 通过上 (被依赖) 下 (依赖方) 游来表达依赖。
- 战术建模
 - 实体 (Entity) : 具有生命周期的连续性和唯一身份标识的可变对象。
 - 值对象 (Value Object) : 只关心属性 (无标识) 的 (不可变) 的对象。
 - 聚合 (Aggregate) : 一组相关对象的集合, 数据修改的单元, 仅根实体可见。
 - 工厂 (Factory) : 领域层负责复杂对象的实例和聚合的创建和装配。
 - 资源 (Repository) : 提供对象的存储和查询, 解耦领域逻辑与持久化。
 - 领域服务 (Domain Service) : 无状态的非实体或值对象的职责 (含资源层的调用)。

参考文档

元数据驱动架构调研: <https://ata.alibaba-inc.com/articles/173637>

元数据驱动Saas架构: <https://ata.alibaba-inc.com/articles/147948>

元数据驱动的业务数据模型设计与管理: <https://ata.alibaba-inc.com/articles/221066>

NBF模型平台: <https://ata.alibaba-inc.com/articles/181025>

阿里云模型平台: <https://ata.alibaba-inc.com/articles/192850#N7a9C1uc>

模型映射工具对比: <https://www.baeldung.com/java-performance-mapping-frameworks>

jsonschema2pojo: <https://www.baeldung.com/java-generate-class-from-json>